

DS Lecture 3: Time in distributed systems

September 18th, 2024

Instructor: Elli Anastasiadi

`ellia@cs.aau.dk`

DEIS
Aalborg University
Denmark



AALBORG UNIVERSITY
DENMARK



Some wrap up from last week

- ▶ I believe you all have group rooms now. yes?
- ▶ **Attention** : Lecture 5 lab not in group rooms! It is an active lab.
- ▶ Teams channel now online (find in Moodle). Please use to post discussions!

Last lecture



How to model a process, types of process failure, types of communication and failures, the two general problem, a tiny proof, timing and time assumptions, performance measures



Last lecture

How to model a process, types of process failure, types of communication and failures, the two general problem, a tiny proof, timing and time assumptions, performance measures

To have a distributed system model we need to pick:

- ▶ A **network** + failure style:
perfect, fair-loss, or arbitrary
- ▶ a **node** failure style:
crash-stop, crash-silent, crash-recovery, Byzantine
- ▶ **Timing:**
Sync or Async.

Agenda



Causes of asynchrony

Physical and digital time

Lamport diagrams

Happens-before

Safety and Liveness

What have we already said about time?



What have we already said about time?



Sync vs Async computation. Without some timing guarantees some things might be impossible (see two generals).



What have we already said about time?

Sync vs Async computation. Without some timing guarantees some things might be impossible (see two generals).

But what causes asynchrony in the first place?

Reasons for Asynchrony (in practice)



Reasons for Asynchrony (in practice)



- Always step one on distributed systems: [take different cases](#) .



Reasons for Asynchrony (in practice)

- ▶ Always step one on distributed systems: [take different cases](#) .
- ▶ Hint: Asynchrony caused by [network](#) or by [nodes](#) .



Reasons for Asynchrony (in practice)

- ▶ Always step one on distributed systems: [take different cases](#) .
- ▶ Hint: Asynchrony caused by [network](#) or by [nodes](#) .
- ▶ Brainstorm session.

Think: When does network latency increase? When do nodes compute for longer than expected?



A not-by-far complete list

Networks might:

- ▶ break physically and require rerouting (google cows and sharks messing up network cables),
- ▶ message loss requires retrying,
- ▶ congestion causes queues
- ▶ "the network" itself might need to re-configure.



A not-by-far complete list

Networks might:

- ▶ break physicaly and require rerouting (google cows and sharks messing up network cables),
- ▶ message loss requires retrying,
- ▶ congestion causes queues
- ▶ "the network" itself might need to re-configure.

Nodes might experience:

- ▶ Operating system scheduler problems.
- ▶ Garbage collection can grab the priority from scheduled tasks.
- ▶ Mistakes.

Q1: Assuming no nodes fail at all, and the network is absolutely perfect, can we still have asynchrony?



A story where nothing went wrong

Back in 2012, in the night from 30 June to 1 July (in europe), a huge amount of online services and systems around the world crashed at the same time.

Servers stop responding.

Airlines can't process payments or reservations or check-ins.

Issue lasts many hours.

There was no RPC (remote procedure call) between the systems that failed.

How do distributed systems use time?



Somohow need to measure time, for example to:



How do distributed systems use time?

Somohow need to measure time, for example to:

1. shcedule taks of cetrain duration, timeouts,



How do distributed systems use time?

Somohow need to measure time, for example to:

1. shcedule taks of cetrain duration, timeouts,
2. hearbeats, retry times, failure detectors,



How do distributed systems use time?

Somohow need to measure time, for example to:

1. schedule tasks of certain duration, timeouts,
2. heartbeats, retry times, failure detectors,
3. record timestamp of events (e.g. in databases)



How do distributed systems use time?

Somohow need to measure time, for example to:

1. shcedule taks of cetrain duration, timeouts,
2. hearbeats, retry times, failure detectors,
3. record timestamp of events (e.g. in databases)
4. measure performance, statistics, . . .



How do distributed systems use time?

Somohow need to measure time, for example to:

1. schedule tasks of certain duration, timeouts,
2. heartbeats, retry times, failure detectors,
3. record timestamp of events (e.g. in databases)
4. measure performance, statistics, ...
5. make data expire (see dig command and certificate example).



How do distributed systems use time?

Somohow need to measure time, for example to:

1. schedule tasks of certain duration, timeouts,
2. heartbeats, retry times, failure detectors,
3. record timestamp of events (e.g. in databases)
4. measure performance, statistics, ...
5. make data expire (see dig command and certificate example).
6. Super important : Determine order of events across nodes.



How do distributed systems use time?

Somohow need to measure time, for example to:

1. schedule tasks of certain duration, timeouts,
2. heartbeats, retry times, failure detectors,
3. record timestamp of events (e.g. in databases)
4. measure performance, statistics, ...
5. make data expire (see dig command and certificate example).
6. Super important : Determine order of events across nodes.

Q: how would you do all of these regardless of distributed systems?

Types of clocks





Types of clocks

- ▶ physical: count amount of time elapsed (in some *unit*)
- ▶ logical: count events (how many requests sent etc).



Types of clocks

- ▶ physical: count amount of time ellapsed (in some *unit*)
- ▶ logical: count events (how many requests sent etc).

For both (in DS!): a clock is something that we can query for a timestamp.

Physical notion of time



Physical notion of time



Physical notion of time



Quartz clocks:

- ▶ cut crystal with lasers to control oscillation frequency
- ▶ Boot laptop = essentially asks to the RTC (= Real Time Clock = a whole other mini-computer) the quartz oscillation stamp (via Piezoelectric effect)
- ▶ count number of cycles in frequency to get elapsed time.

Great!



Great!



...not so fast...

Great!



... not so fast...

- ▶ Quartz crystals are physical things and might have slight differences.

Great!



... not so fast...

- ▶ Quartz crystals are physical things and might have slight differences.
- ▶ Frequency depends on temperature!!

Great!



... not so fast...

- ▶ Quartz crystals are physical things and might have slight differences.
- ▶ Frequency depends on temperature!!
- ▶ We measure drift of clocks in **parts per milion** (ppm).
- ▶ $1 \text{ ppm} = 1 \text{ microsecond/second} = 32\text{s/year}$.
- ▶ Most computer clocks okay with about 50ppm.

Other options



- ▶ Atomic clocks (very very expensive)
- ▶ GPS (satellites use atomic clocks, get signal from them and calculate)
- ▶ "ask the internet"

Q: How to turn these clocks into something that can tell us times?



We start involving the sun

Solar time (GMT): It is noon, when sun is at place XXX when we stand at place YYY. 1 day is time between two noons.

International Atomic Time (TAI): 1 day = ZZZ number of oscillations of caesium-133 in resonant frequency.

Problem: These two do not match (year round).

Compromise: UTC (coordinated universal time) = TAI + corrections.



Doing the correction (for real life)

Leap seconds : Two times per year (June 30 and 31 December at 23:59:59 UTC) we get:

- ▶ to skip one second (negative leap second) - 23:59:58 followed by 00:00:00,
- ▶ nothing special,
- ▶ to add one second (positive leap second) - 23:59:59 followed by 23:59:60.

Doing the correction (for real life)

Leap seconds : Two times per year (June 30 and 31 December at 23:59:59 UTC) we get:

- ▶ to skip one second (negative leap second) - 23:59:58 followed by 00:00:00,
- ▶ nothing special,
- ▶ to add one second (positive leap second) - 23:59:59 followed by 23:59:60.



What your computer tells you



The two most common time representations:

What your computer tells you

The two most common time representations:

- ▶ **Unix time**: number of seconds since 1 January 1970 00:00:00 UTC.
What we call the "epoch". **This does not count leap seconds**
- ▶ ISO 8601: timestamps relative to UTC

What your computer tells you

The two most common time representations:

- ▶ **Unix time**: number of seconds since 1 January 1970 00:00:00 UTC.
What we call the "epoch". **This does not count leap seconds**
- ▶ ISO 8601: timestamps relative to UTC
- ▶ Computers have conventions to transpose each one of these to another.

What your computer tells you

The two most common time representations:

- ▶ **Unix time:** number of seconds since 1 January 1970 00:00:00 UTC.
What we call the "epoch". **This does not count leap seconds**
- ▶ ISO 8601: timestamps relative to UTC
- ▶ Computers have conventions to transpose each one of these to another.
Q: How does software deal with the leap seconds in the above conversion?

Break

Network Time protocol



Like we said: computers have quartz clocks always running which are subject to **clock drift**.



Network Time protocol

Like we said: computers have quartz clocks always running which are subject to **clock drift**.

Solution: Periodically get the time from server (with better clocks).

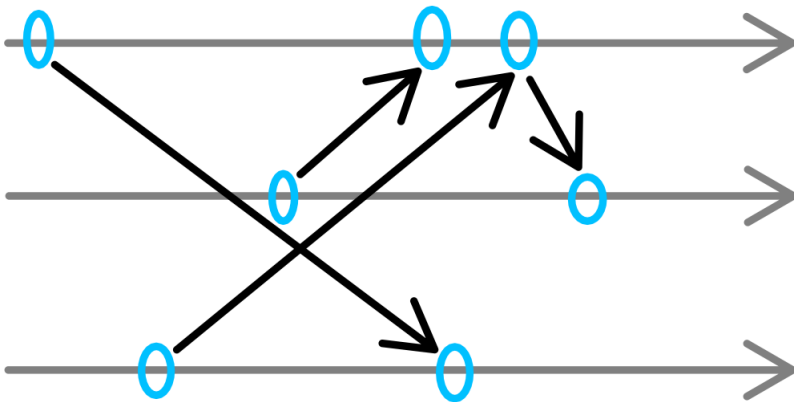
Protocols: NTP, and P (= precision) TP.

On ubuntu: `systemctl status systemd-timesyncd`

Intermission: Drawing time

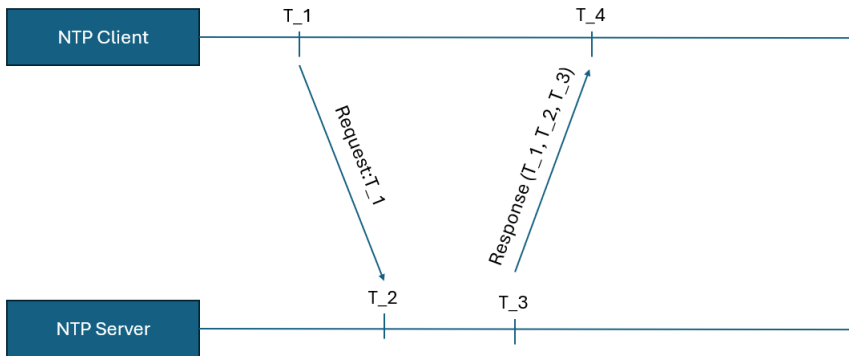


Intermission: Drawing time

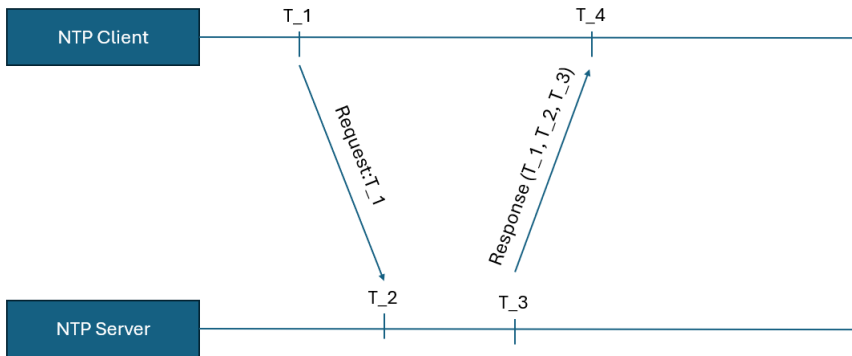


A Lamport diagram (from the same Lamport who was complaining about computers failing far away).

Lamport diagram with NTP



Lamport diagram with NTP



trip network times: $\delta = (T_4 - T_1) - (T_3 - T_2)$

Round

NTP continued



Client knows δ + Assumes symmetric network delay.

NTP continued



Client knows δ + Assumes symmetric network delay.

When should we receive the response?

NTP continued



Client knows δ + Assumes symmetric network delay.

When should we receive the response? $T_3 + (\delta \div 2)$.

NTP continued



Client knows δ + Assumes symmetric network delay.

When should we receive the response? $T_3 + (\delta \div 2)$.

Clock skew θ is difference between client and server clocks.

NTP continued



Client knows δ + Assumes symmetric network delay.

When should we receive the response? $T_3 + (\delta \div 2)$.

Clock skew θ is difference between client and server clocks. $T_3 + (\delta \div 2) - T_4$

NTP continued

Client knows δ + Assumes symmetric network delay.

When should we receive the response? $T_3 + (\delta \div 2)$.

Clock skew θ is difference between client and server clocks. $T_3 + (\delta \div 2) - T_4$

Depending on how large θ is the client can adapt (slew, reset, or ignore).

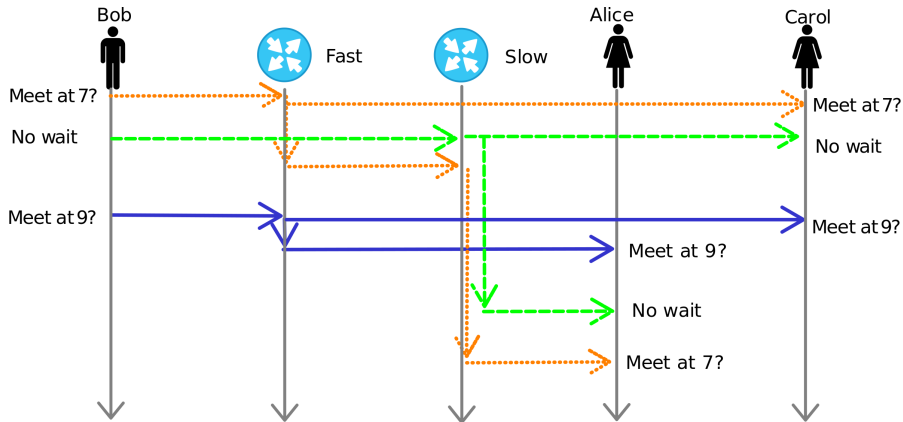
- ▶ if $\theta < 125ms$ then slew.
- ▶ if $125 \leq \theta < 1000s$ then jump.
- ▶ else ignore.

Lesson

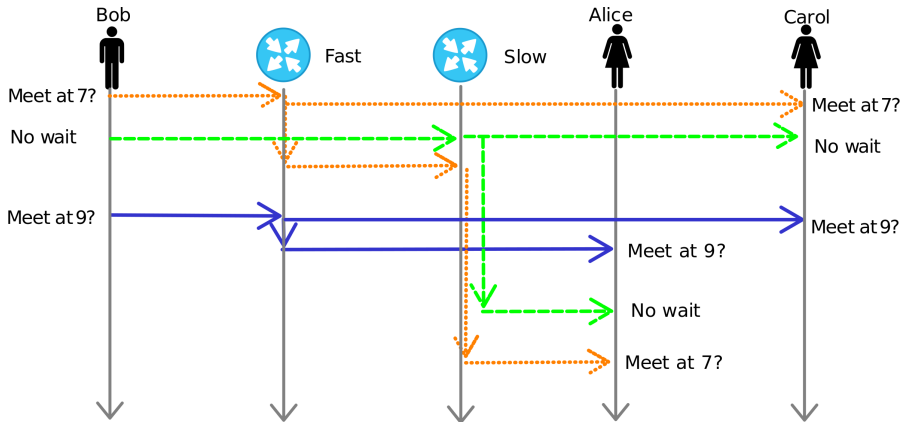
Monotonic and time-of-day clocks

- ▶ Time-of-day:
 - ▶ Time since fixed date (UNIX time)
 - ▶ Can jump, must fix leap seconds
 - ▶ Can compare timestamps across nodes
- ▶ Monotonic
 - ▶ Time since arbitrary point (F.e. computer boot).
 - ▶ Does not jump and has near-fixed rate.
 - ▶ Good for measuring elapsed time.

Logical clocks



Logical clocks

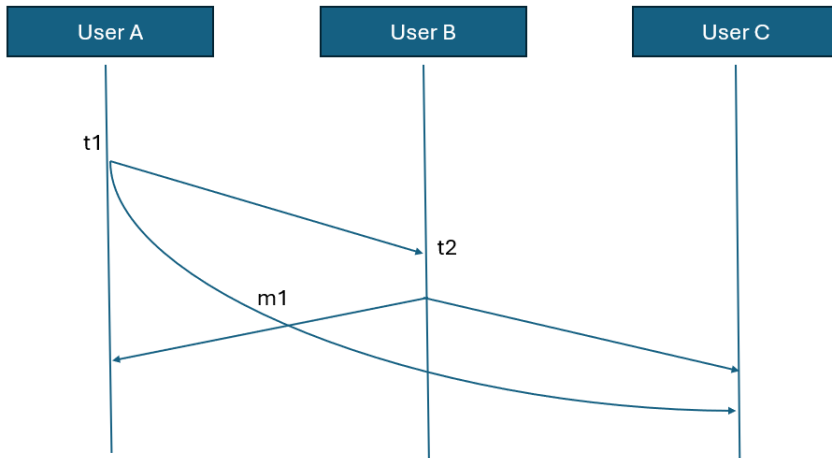


Alice receives "meet at 9?" and "meet at 7?" last, even though the "meet at 7?" **happened before** "meet at 9?".

Can we fix this with timestamps?



Can we fix this with timestamps?





The happens-before relation

(We call an event something that happens in one node *atomically*)

The happens-before relation

(We call an event something that happens in one node *atomically*)

We say event a happens before event b (write $a \implies b$ or (a, b)) iff:

- ▶ a and b take place in the same node, and a occurs before b in the nodes local execution, or
- ▶ a is a send event, and b is the corresponding receive, or
- ▶ There exists c such that both $a \implies c$ and $c \implies b$.

The happens-before relation

(We call an event something that happens in one node *atomically*)

We say event a happens before event b (write $a \implies b$ or (a, b)) iff:

- ▶ a and b take place in the same node, and a occurs before b in the nodes local execution, or
- ▶ a is a send event, and b is the corresponding receive, or
- ▶ There exists c such that both $a \implies c$ and $c \implies b$.

The happens-before relation is only a partial order. I.e. it can be the case that **neither** $a \implies b$, nor $b \implies a$.

The happens-before relation

(We call an event something that happens in one node *atomically*)

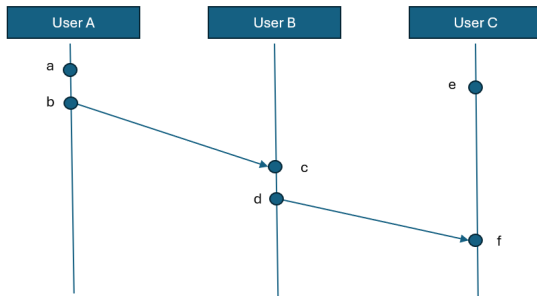
We say event a happens before event b (write $a \implies b$ or (a, b)) iff:

- ▶ a and b take place in the same node, and a occurs before b in the nodes local execution, or
- ▶ a is a send event, and b is the corresponding receive, or
- ▶ There exists c such that both $a \implies c$ and $c \implies b$.

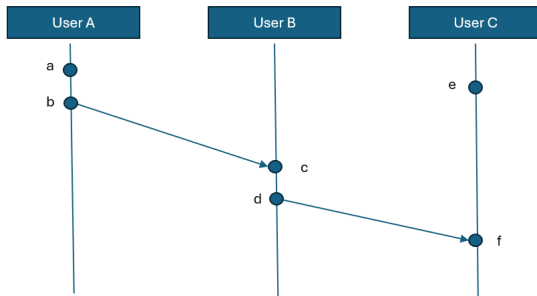
The happens-before relation is only a partial order. I.e. it can be the case that **neither** $a \implies b$, nor $b \implies a$.

In this case we say a and b are concurrent (and we write $a || b$).

Example



Example



- ▶ $a \implies b, c \implies d$, and $e \implies f$ due to local process order.
- ▶ $b \implies c$ and $d \implies f$ due to messages send and receive.
- ▶ item $a \implies c, a \implies d, a \implies f, \dots$

Causality



Notion of causality is borrowed from physics.

- ▶ (General belief): Information does not travel faster than the speed of light.

Notion of causality is borrowed from physics.

- ▶ (General belief): Information does not travel faster than the speed of light.
- ▶ If events a and b happen far apart but close in time then they are not causally related.
- ▶ Otherwise (if light of one event reaches the other) then a might influence b .

Notion of causality is borrowed from physics.

- ▶ (General belief): Information does not travel faster than the speed of light.
- ▶ If events a and b happen far apart but close in time then they are not causally related.
- ▶ Otherwise (if light of one event reaches the other) then a might influence b .
- ▶ In DS: instead of light we use messages.



How can we use the hb relation and causality?

Brainstorm

How can we use the hb relation and causality?

Brainstorm

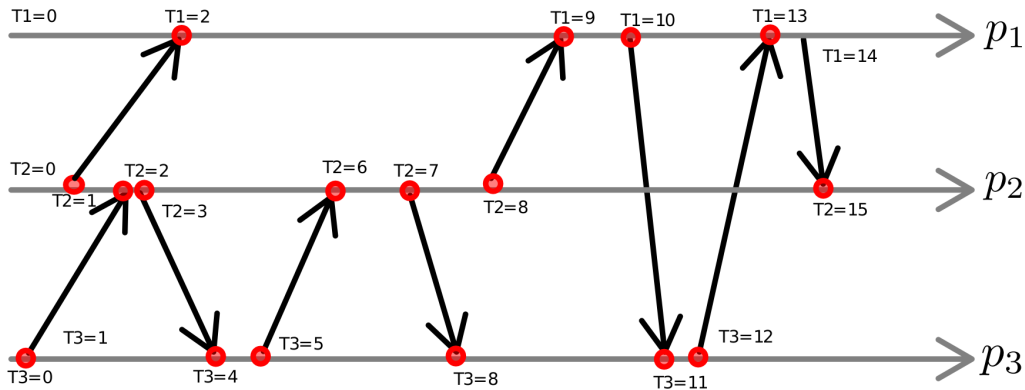
Some answers:

- ▶ For debugging.
- ▶ To realise our router is faulty.
- ▶ To realise some clock thing is going bad.
- ▶ To realise our code is bad (we will get back to this).
- ▶ To optimize! :)

Lamport clocks



Lamport clocks



Lamport clocks Algorithm





Lamport clocks Algorithm

- ▶ Each node/process gets its own local clock variable t , and initializes at 0.
- ▶ Every time a node gets an event they increment t .
- ▶ If the event is a send of message m , the node sends (t, m) (after incrementing t).
- ▶ If the node is receiving a message then the node updates their clock to the maximum of their local clock and the received value, and then increments.

Properties



Properties



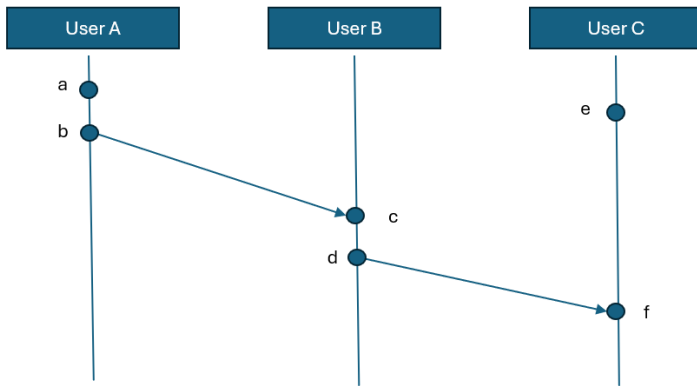
If $a \implies b$ then $L(a) < L(b)$.

Properties

If $a \implies b$ then $L(a) < L(b)$.

If $L(a) < L(b)$ it is not the case that $a \implies b$.

Example



Safety and Liveness





Safety and Liveness

Usually: To make a distributed algorithm we want to satisfy properties in all possible executions of the algorithm, covering:

- ▶ all possible sequences of steps executed by the processes according to the algorithm
- ▶ scheduling of these steps remains outside the control of the processes and depends on the global scheduler.



Safety and Liveness

Usually: To make a distributed algorithm we want to satisfy properties in all possible executions of the algorithm, covering:

- ▶ all possible sequences of steps executed by the processes according to the algorithm
- ▶ scheduling of these steps remains outside the control of the processes and depends on the global scheduler.

Usual types:

- ▶ Safety : properties state that the algorithm should not do anything wrong.

Safety and Liveness

Usually: To make a distributed algorithm we want to satisfy properties in all possible executions of the algorithm, covering:

- ▶ all possible sequences of steps executed by the processes according to the algorithm
- ▶ scheduling of these steps remains outside the control of the processes and depends on the global scheduler.

Usual types:

- ▶ **Safety** : properties state that the algorithm should not do anything wrong. these can be violated at some time t and never be satisfied again after that time.
- ▶ **Liveness** : eventually something good happens.

Safety and Liveness

Usually: To make a distributed algorithm we want to satisfy properties in all possible executions of the algorithm, covering:

- ▶ all possible sequences of steps executed by the processes according to the algorithm
- ▶ scheduling of these steps remains outside the control of the processes and depends on the global scheduler.

Usual types:

- ▶ **Safety** : properties state that the algorithm should not do anything wrong. these can be violated at some time t and never be satisfied again after that time.
- ▶ **Liveness** : eventually something good happens. these can be satisfied at some time t and never be violated again after that time.

Formaly





Formaly

Safety definition

A safety property is a property such that, whenever it is violated in some execution E of an algorithm, there is a partial execution E' of E (a prefix) such that the property will be violated in any extension of E' .



Formaly

Safety definition

A safety property is a property such that, whenever it is violated in some execution E of an algorithm, there is a partial execution E' of E (a prefix) such that the property will be violated in any extension of E' .

Liveness definition

A liveness property is a property of a distributed system execution such that, for any partial execution E' there are continuations E of E' where the property can be satisfied.

Examples



safety property: The property of perfect links: no process should receive a message unless this message was indeed sent.



Examples

safety property: The property of perfect links: no process should receive a message unless this message was indeed sent.

liveness property: Also in perfect links: if a correct process sends a message to a correct destination process, then the destination process should eventually deliver the message.